

רשתות RF, SDR, MANET ו-BNET

ובדיקות אוטומציה מעליהן

סיכום מסודר של כל החומר: יסודות רדיו, יחידות מידה, רשתות אד-הוק, BNET של רפאל ברמה הפומבית, איך אפליקציית ניטור מתקשרת עם הרדיו, תרשימים, כלים, ארכיטקטורת בדיקות, רשתות לעומק, ICD ופרוטוקולים בינאריים, מישור ניהול, סנכרון זמן, אמוציית רשת, ותהליך הבדיקות בתעשייה הביטחונית.

כל המידע על BNET במסמך הזה מבוסס על פרסומים פומביים בלבד. פרטים על ממשקים, פרוטוקולים ו-waveform אמיתיים מגיעים מהתיעוד הפנימי בעבודה.

תוכן עניינים

1. יסודות RF
2. יחידות מידה
3. רשת RF – טופולוגיות וגישה לערוץ
4. SDR – רדיו מוגדר תוכנה
5. רשתות Ad-hoc ו-MANET
6. BNET של רפאל – מה ידוע בפומבי
7. BNET מול אפליקציית ניטור
8. תרשימים: איך אפליקציה-קליינט עובדת מול BNET
9. ארגז הכלים של מפתחת בדיקות
10. ארכיטקטורה של מסגרת בדיקות
11. רשתות לעומק – מה צריך לדעת כדי לבדוק
12. תרחישי בדיקה, סדר עדיפויות ומילון מושגים
13. ICD ופענוח פרוטוקולים בינאריים
14. מישור הניהול: SNMP, syslog ומצבי רדיו
15. סנכרון זמן ומתאם לוגים
16. בניית רשת רב-צמתית על מחשב אחד
17. תהליך הבדיקות בתעשייה הביטחונית
18. Multicast לעומק
19. סוקטים לעומק ודפוסי אמינות
20. בדיקות ממושכות, הזרקת תקלות וקנה מידה
21. אוטומציה ממשק משתמש שולחני
22. בדיקות מבוססות מודל ותכונות
23. נתוני תמונת מצב: CoT, PLI ו-TAK
24. Waveforms, קישורי נתונים ותקני NATO
25. לוחמה אלקטרונית וניהול ספקטרום
26. תקני סביבה ו-EMC
27. נושאים משלימים ותוכנית תרגול

1 יסודות RF

שורה תחתונה: רשת RF היא כל רשת שמעבירה מידע באוויר בגלי רדיו. רשת SDR היא רשת כזו שבה הרדיו ממומש בעיקר בתוכנה, ולכן אפשר לשנות תדר, אפנון ופרוטוקול בלי להחליף חומרה. ברשתות טקטיות (כמו BNET) משלבים SDR עם MANET – רשת משמרגנת את עצמה בלי תשתית קבועה.

תדר – Frequency

מספר המחזורים בשנייה של הגל. תדר נמוך מגיע רחוק וחודר מכשולים טוב יותר; תדר גבוה נותן יותר רוחב פס אבל טווח קצר יותר.

רוחב פס – Bandwidth

טווח התדרים שהאות תופס. יותר רוחב פס מאפשר יותר ביטים בשנייה.

אפנון – Modulation

הדרך לשתול ביטים בתוך הגל, על ידי שינוי משרעת, תדר או פאזה. דוגמאות: BPSK, QPSK, QAM. אפנון "צפוף" (כמו 64QAM) מעביר יותר מידע אבל דורש אות נקי יותר.

יחס אות לרעש – SNR

כמה האות חזק ביחס לרעש, ביחידות dB. המדד הכי חשוב לאיכות קישור.

עוצמת אות נקלט – RSSI

כמה הספק מגיע למקלט, בדרך כלל ב-dBm. הרדיו מדווח עליו כמעט תמיד, והוא מדד מרכזי בבדיקות.

ריבוי נתיבים ודעיכה – Multipath / Fading

האות מגיע במקביל בכמה מסלולים (החזרות מקירות ומהקרקע) ולכן מתחזק או נחלש. זו הסיבה שהאיכות משתנה גם כשלא זזים.

תקציב קישור – Link Budget

החשבון שקובע אם קישור יעבוד:

$$Prx = Ptx + Gtx + Grx - Losses$$

אם ההספק הנקלט גבוה מרגישות המקלט (Sensitivity) – הקישור עובד.

אובדן מסלול במרחב חופשי – FSPL

$$FSPL(dB) = 20 \cdot \log(d_km) + 20 \cdot \log(f_MHz) + 32.44$$

כלל אצבע: כל 6 dB נוספים מכפילים את הטווח בתנאים אידיאליים.

שורה תחתונה: היחידה המבלבלת היא דציבל. היא לא מודדת כמות אלא יחס, בסולם לוגריתמי. ארבעה כללים מספיקים כדי לקרוא כל מספר.

תדר

יחידה	משמעות	דוגמה
הרץ - Hz	מחזור אחד בשנייה	-
קילו-הרץ - kHz	אלף מחזורים בשנייה	-
מגה-הרץ - MHz	מיליון מחזורים בשנייה	רדיו FM סביב 100 MHz
גיגה-הרץ - GHz	מיליארד מחזורים בשנייה	Wi-Fi על 2.4 ו-5 GHz

הספק

וואט - W

יחידת ההספק רגילה. רדיו טקטי משדר בדרך כלל כמה וואטים.

מיליוואט - mW

אלפית וואט. נתב Wi-Fi ביתי משדר בערך 100 mW.

דציבל - dB

משתמשים בו כי ברדיו ההבדלים עצומים: אות יכול להיחלש פי מיליארד בדרך למקלט.

דציבל-מיליוואט - dBm

אותו רעיון, אבל ביחס לנקודת ייחוס קבועה של 1 mW, ולכן זו כן יחידת כמות.

ערך	הספק	הערה
dBm 0	1 mW	נקודת הייחוס
dBm 20	100 mW	נתב Wi-Fi
dBm 30	1 W	-
מספר שלילי	פחות מ-1 mW	כאן נמצאים האותות הנקלטים

איך לקרוא RSSI (קרוב יותר לאפס = חזק יותר)

ערך	איכות
מינוס 50 dBm	חזק מאוד
מינוס 70 dBm	טוב
מינוס 85 dBm	חלש
מינוס 95 dBm ומטה	סף ניתוק ברוב המכשירים

שינוי	משמעות
פלוס 3 dB	פי 2
פלוס 10 dB	פי 10
מינוס 3 dB	חצי
מינוס 10 dB	עשירית
20 dB	פי 100 (10 כפול 10)
30 dB	פי 1,000
13 dB	פי 20 (10 ועוד 3 = 10 כפול 2)

דוגמה: SNR של 10 dB - האות חזק פי 10 מהרעש. SNR של 0 dB - האות והרעש שווים, זהו כבר קשה מאוד לפענח.

קצב נתונים וזמן

ביטים לשנייה - bps / kbps / Mbps / Gbps

כמה ביטים עוברים בשנייה. שימי לב: Mbps עם b קטנה = ביטים. MB עם B גדולה = בייטים. בייט = 8 ביטים.

מילישנייה - ms

אלפית שנייה. בה מודדים השהיה. ברשת טקטית רב-קפיצות, מאות מילישניות זה רגיל.

שיעור שגיאות ביטים - BER

החלק היחסי של הביטים שהתקבלו שגויים. BER של 0.001 = ביט שגוי אחד מכל אלף.

3 רשת RF - טופולוגיות וגישה לערוץ

נקודה-לנקודה - Point-to-Point

שני צמתים בלבד, למשל קישור מיקרוגל.

כוכב - Star

כולם מדברים דרך מרכז (נתב Wi-Fi, אנטנה סולרית). פשוט, אבל אם המרכז נופל - הכול נופל.

Mesh

כל צומת יכול להעביר הלאה מידע של אחרים. אין נקודת כשל יחידה.

גישה לערוץ - MAC

מי משדר ומתי, כי כולם חולקים את אותו "אוויר".
TDMA - חלוקה לחריצי זמן קבועים. צפוי, דטרמיניסטי, נפוץ בצבא.
CSMA - מקשיבים ומשדרים כשהערוץ פנוי, כמו Wi-Fi.

חצי דופלקס – Half-Duplex

רוב מכשירי הרדיו לא יכולים לשדר ולקלוט בו-זמנית על אותו תדר. לכן כל קפיצה ברשת מש מורידה תפוקה.

4 SDR – רדיו מוגדר תוכנה

ברדיו קלאסי האפנון, הסינון והפענוח נעשים ברכיבי חומרה ייעודיים. ב-SDR רק החזית האנלוגית נשארת חומרה, וכל השאר קורה בקוד.

Antenna → RF front-end → ADC/DAC → FPGA/DSP → General CPU → Application
(amp, filter, mixer) (signal processing) (protocol, routing)

שרשרת העיבוד ב-SDR: רק השלב הראשון אנלוגי

דגימות I/Q – IQ Samples

כל דגימה היא מספר מרוכב (Quadrature-In-phase) שמייצג גם משרעת וגם פאזה. כל עיבוד האותות ב-SDR עובד על זרם כזה.

ממיר אנלוגי-דיגיטלי – ADC

הופך את האות האנלוגי לדגימות מספריות.

ארכיטקטורת תקשורת תוכנה – SCA

תקן צבאי שמגדיר איך בונים waveforms ניידים בין פלטפורמות SDR.

צורת גל – Waveform

חבילת התוכנה שמגדירה איך הרדיו משדר: אפנון, קידוד, MAC ופרוטוקול. ב-SDR מחליפים waveform כמו שמחליפים אפליקציה.

יתרונות SDR

- עדכון תוכנה במקום החלפת חומרה.
- כמה פרוטוקולים באותו מכשיר, כולל תאימות לרדיואים ישנים.
- התאמה דינמית לתנאי הערוץ.

יכולות נוספות ברדיו טקטי

ריבוי אנטנות – MIMO

כמה אנטנות בשידור ובקליטה מנצלות את ריבוי הנתיבים לטובת תפוקה ויציבות.

קפיצות תדר – Frequency Hopping

החלפת תדר מאות או אלפי פעמים בשנייה לפי רצף סודי. מקשה על שיבוש ועל האזנה.

רדיו קוגניטיבי – Cognitive Radio

רדיו שסורק את הספקטרום ובוחר לבד תדר פנוי או נקי משיבוש.

תרגיל פייתון: QPSK, רעש ו-BER

```
import numpy as np

rng = np.random.default_rng(seed=0)
num_symbols = 1000
snr_db = 10

# Map random bit pairs to QPSK constellation points
bits = rng.integers(0, 2, size=(num_symbols, 2))
symbols = ((2 * bits[:, 0] - 1) + 1j * (2 * bits[:, 1] - 1)) / np.sqrt(2)

# Add complex Gaussian noise for the requested SNR
noise_power = 10 ** (-snr_db / 10)
noise = np.sqrt(noise_power / 2) * (
    rng.standard_normal(num_symbols) + 1j * rng.standard_normal(num_symbols)
)
received = symbols + noise

# Hard-decision demodulation and bit error rate
decoded = np.column_stack((received.real > 0, received.imag > 0)).astype(int)
ber = np.mean(decoded != bits)
print(f"SNR={snr_db} dB, BER={ber:.4f}")
```

להריץ עם SNR של 5, 10 ו-10 ולראות איך ה-BER צונח. זה בדיוק הקשר בין איכות קישור לאמינות הרשת.

שורה תחתונה: רשת אד-הוק היא רשת שבה המכשירים מדברים ישירות זה עם זה, בלי נתב או אנטנה מרכזית. MANET היא רשת אד-הוק שבה המכשירים גם זזים כל הזמן, ולכן הרשת צריכה לגלות מחדש מי מחובר למי ולחשב מסלולים אוטומטית.

רשת אד-הוק – Ad-hoc Network

אין מרכז. כל מכשיר מדבר ישירות עם מי שבטוחו שלו. בלטינית:
 "לצורך העניין הזה" – רשת שנוצרת במקום, כשצריך. דוגמאות:
 Bluetooth, Wi-Fi Direct.

רשת מבוססת תשתית – Infrastructure Network

כמו ברשת הביתית: כל המכשירים מדברים דרך הנתב. הנתב כבה – אין רשת.

MANET (Mobile Ad-hoc Network) – רשת אד-הוק ניידת

רשת אד-הוק עם שתי תוספות: (1) כל הצמתים זזים, ולכן מבנה הרשת משתנה כל הזמן. (2) כל צומת משמש גם כנתב ומעביר הלאה הודעות של אחרים.

מושגי יסוד

צומת – Node

מכשיר ברשת.

קישור - Link

הקשר התקשורתי בין שני צמתים. לכל קישור יש איכות משלו.

טופולוגיה – Topology

המבנה של הרשת כרגע: מי מחובר למי.

Multi-hop – ריבוי קפיצות

מידע עובר דרך צמתי ביניים עד שהוא מגיע ליעד.

ריפוי עצמי – Self-healing

כשצומת נופל, הרשת מוצאת מסלול חלופי בעצמה.

Multi-hop:

A <----> B <----> C
(A and C out of range)

$$A \rightarrow B \rightarrow C$$

Self-healing:

```
A → B → C      B fails
  ↓
A → D → C      alternate route
```

איך MANET עובדת

גילוי שכנים – Neighbor Discovery

כל רדיו משדר כל כמה שניות הודעת Hello. מי ששומע אותה יודע שהשולח הוא שכן ישיר.

Proactive Routing (OLSR) – ניתוב פרואקטיבי

כל צומת מחזיק מפה מלאה של הרשת כל הזמן. המסלול מוכן מיד, אבל יש הרבה תעבורת ניהול ברקע.

ניתוב ריאקטיבי – Reactive Routing (AODV)

מחפשים מסלול רק כשצריך: A שולח לכולם בקשה (RREQ), הבקשה מתפשטת עד C, ו-C מחזיר תשובה במסלול ההפוך (RREP). פחות תעבורה, אבל עיכוב בהודעה הראשונה.

ניתוב היברידי - Hybrid Routing

פרואקטיבי לשכנים קרובים, ריאקטיבי ליעדים רחוקים.

אתגרים ב-MANET

אתגר	הסבר
טופולוגיה משתנה	קישורים נוצרים ונשברים כל הזמן, מסלולים מתיישנים מהר.
רוחב פס מוגבל	כולם חולקים ערוץ אחד, ותעבורת הניתוב "אוכלת" ממנו.
הצומת הנסתר – Hidden Terminal	A ו-C לא שומעים זה את זה ומשדרים ל-B יחד – התנגשות.
סערת שידורים – Broadcast Storm	כל צומת מעביר כל בקשה והרשת מוצפת.
אבטחה	אין שרת מרכזי לאימות; צומת עוין יכול להזיק.
אנרגיה	העברת הודעות של אחרים צורכת סוללה.

סוגים נפוצים

VANET – רכבים, **FANET** – רחפנים, **Tactical MANET** – חיילים, כלי רכב ומפקדות בשטח, זה העולם של **BNET**.

שורה תחתונה: BNET הוא משפחת רדיו טקטי שמשלבת SDR (הרדיו ממומש בתוכנה) עם רשת MANET מעליו. הוא לוקח את העיקרון של רשת אד-הוק ניידת והופך אותו למוצר קרבי שמעביר קול, וידאו ודאטה בתנועה, בלי תשתית.

ההגדרה

רפאל מגדירה את BNET כ-SDR MANET IP לפעילות טקטית, עם ניצול ספקטרום יעיל ויכולות קוגניטיביות. הוא מיועד לקישוריות מהירה ובהשגחה נמוכה לקול, וידאו ודאטה בתנועה – לפלטפורמות קרקעיות ואוויריות, מפקדות וחיילים רגליים.

רכיב בשם	המשמעות
IP	האפליקציה מעל הרדיו רואה רשת רגילה: כתובות IP, סוקטים, UDP/TCP.
MANET	רשת אד-הוק ניידת עם ריבוי קפיצות וארגון עצמי.
SDR	אפנון ופרוטוקולים ממומשים בתוכנה, ולכן ניתנים לעדכון.

הקשר ל-Ad-hoc

בלי תשתית – No Infrastructure מיועד למצבים של ניידות גבוהה בלי תשתית זמינה. הרדיואים עצמם הם הרשת.	רשת שטוחה גדולה – Flat Scalable Network לפי החומר הפומבי, תומך ברשתות "שטוחות" של אלפי משתתפים בשטח משתנה: הרים, עיר, יער, מדבר. זה המבחן האמיתי של MANET.
ריפוי עצמי – Self-healing הרשת מתקנת את עצמה כשצמתים נופלים או זזים.	צירוף ומיזוג מהירים – Merge & Join ב-MANET הרשת יכולה להתפצל לשתיים (Partition) ואז להתאחד. Merge = איחוד תת-רשתות; Join = צירוף צומת חדש. המהירות שלהם היא פרמטר ביצועים שנבדק.
רשת אחת לכולם – Single Seamless Network יחידות יבשה, ים ואוויר ברשת אחת.	

מה מייחד את BNET טכנולוגית

קליטה רב-ערוצית – Multi Channel Reception (MCR) רדיו יחיד עם אנטנה יחידה קולט ומעבד במקביל מאות MHz וממזג אותם לרשת IP אחת. ברדיו MANET רגיל כולם חולקים ערוץ אחד; כאן אפשר לפרוס משתמשים על ערוצים שונים בלי לפרק את הרשת. לכן הטענה ליותר מ-1000 משתמשים.	מודעות ספקטרום – Spectrum Awareness / Cognitive כל רדיו חש את הספקטרום באופן רציף ובוחר איפה לעבוד.
עמידות לשיבוש ועצמאות מ-GPS – Anti-jam, GPS-independent רדיואים רבים משתמשים ב-GPS לסנכרון זמן, ובשדה קרב GPS משובש לעיתים קרובות.	תאימות לתקן – SCA 2.2.2 התקן הצבאי ל-waveforms ניידים ב-SDR.
ארכיטקטורה פתוחה – Open IP Architecture אינטגרציה עם אפליקציות קיימות ועתידיות דרך IP. זו הנקודה שמחברת לאפליקציית הניטור: היא לא צריכה לדעת דבר על אפנון או קפיצות תדר.	תאימות לאחור – Interoperability הדגמים תואמים לרדיואים מדור קודם – יתרון ישיר של SDR.

משפחת המוצרים

דגם	שימוש
BNET HH	מחזיק יד, לחייל רגלי
BNET MP	מנשא גב
BNET V	רכבי (כ-15 ק"ג)
BNET AR	מוטס (כ-11 ק"ג)
BNET Nano	הקטן ביותר (כ-60 גרם), גרסאות לכטב"מים ולחיל רגלים

שורה תחתונה: BNET הוא מכשיר תקשורת – צומת ברשת. אפליקציית ניטור היא תוכנה שמקבלת ממנו מידע ומציגה למפעיל את מצב הרשת. האפליקציה לא מתקשרת "עם BNET ועם Ad-hoc" כשני דברים נפרדים: היא מתקשרת עם BNET, ו-BNET הוא חלק מרשת Ad-hoc.

בעבודה אנשים אומרים "בינט" ולפעמים מתכוונים לכל המערכת. כדאי לברר בימים הראשונים איך קוראים לכל רכיב אצלכם.

קלט, עיבוד ופלט של BNET

שלב	מה קורה
קלט	אות RF ממכשירי BNET אחרים, ונתוני IP ממערכת חיצונית דרך ממשק נתונים (Ethernet / USB).
עיבוד	קליטה, עיבוד תקשורת, ניתוב IP משולב והעברה ברשת.
פלט	שידור RF לצמתים אחרים, נתוני IP למחשב/אפליקציה, ומידע לניהול וניטור.

ארבעת תפקידי אפליקציית הניטור

1. מקבלת מידע מהרדיו.
2. מעבדת: מה השתנה, האם יש תקלה, האם נתון חצה סף.
3. מציגה: טבלאות, מפה, גרפים, סטטוסים, התראות.
4. לפעמים מאפשרת פעולה – אבל ניטור (Monitoring: "מה קורה?") שונה משליטה (Control: "תעשה עכשיו").

חידוד 1: מישור נתונים מול מישור ניהול

מישור הניהול – Management Plane

המידע על הרשת: מי מחובר, איכות קישורים, מצב הרדיו, התראות.

מישור הנתונים – Data Plane

התעבורה עצמה: קול, וידאו וחבילות IP שעוברות מצומת לצומת.

אפליקציית ניטור יושבת כמעט כולה על מישור הניהול. תרחיש בדיקה חשוב: כשמישור הנתונים עמוס (וידאו כבד), האם דיווחי הניהול עדיין מגיעים בזמן?

חידוד 2: האפליקציה מתחברת לרדיו אחד

האפליקציה מחוברת פיזית לרדיו מקומי אחד. הוא מחזיק תמונת מצב של כל הרשת, כי כל הצמתים מחליפים מידע ניתוב ברקע. לכן מה שהאפליקציה רואה הגיע דרך הרשת – ועלול להיות מיושן. "כמה זמן עד שהאפליקציה יודעת שצומת נפל" היא שאלת בדיקה אמיתית.

חידוד 3: תשאול מול דחיפה

דחיפת אירועים – Push / Notification

הרדיו מודיע יזום. מהיר, אבל אם הודעה אובדת – המצב בתצוגה שגוי.

תשאול תקופתי – Polling

האפליקציה שואלת כל כמה שניות. פשוט, אבל מאחר בזיהוי שינויים.

רוב המערכות משלבות את שניהם. באג קלאסי: אירוע אובד, והתצוגה מתקנת רק בסנכרון הבא.

חידוד 4: קישורים אי-סימטריים

A יכול לשמוע את B היטב בזמן ש-B שומע את A גרוע (הפרשי הספק, אנטנות, רעש). שאלות בדיקה: האפליקציה מציגה ערך אחד לקישור או ערך לכל כיוון? אם אחד – מאיזה כיוון?

שרשרת המושגים

BNET → Node → Link → Topology → Multi-hop → Routing → Link Quality → MANET
→ Data / Management Plane → Convergence Time → State Consistency

עקביות מצב – State Consistency

האם מה שהאפליקציה מציגה תואם למציאות. זו השאלה המרכזית של כל הבדיקות.

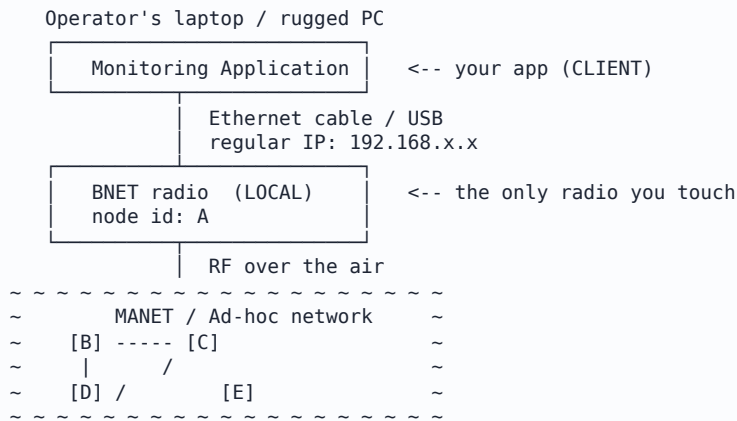
זמן התכנסות – Convergence Time

כמה זמן לוקח לרשת להתייצב על טופולוגיה נכונה אחרי שינוי.

8 תרשימים: איך אפליקציה-קליינט עובדת מול BNET

שורה תחתונה: האפליקציה היא קליינט של רדיו אחד בלבד – הרדיו המקומי שמחובר אליה בכבל. היא מדברת איתו IP רגיל. הרדיו הזה מחזיק את תמונת המצב של כל רשת האד-הוק. האפליקציה לעולם לא "מתחברת" לרדיואים המרוחקים – היא קוראת מה שהרדיו המקומי כבר יודע עליהם.

תרשים 1: מי מחובר למי פיזית



הקו היחיד שהאפליקציה מכירה הוא הכבל למעלה. כל השאר קורה מתחת לפני השטח.

תרשים 2: למה הרדיו המקומי יודע על כולם

Step 1: radios exchange routing/topology info continuously (background)
[A] <--hello--> [B] <--hello--> [C]

Step 2: every radio builds its own view of the network
Radio A's internal table:

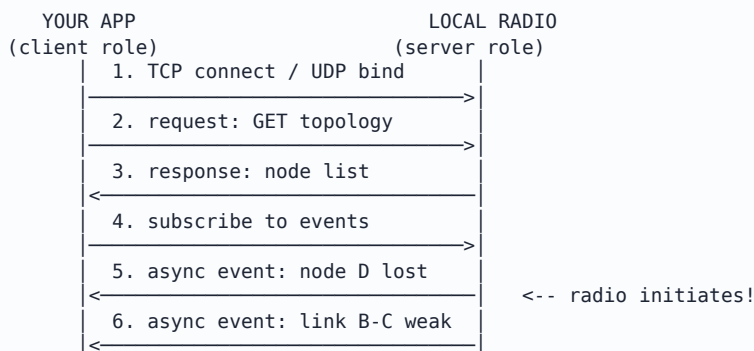
node	next hop	hops	RSSI
B	B	1	-62dBm
C	B	2	n/a
D	B	2	n/a

Step 3: the app just reads that table

App --- "give me topology" ---> Radio A
App <--- table contents ----- Radio A

ל-B יש קפיצה אחת ולכן יש RSSI אמיתי. ל-C יש שתי קפיצות ואין RSSI, כי A לא שומע אותו ישירות. טעות נפוצה באפליקציות ניטור: להציג RSSI לצומת שאינו שכן ישיר. שווה לבדוק.

תרשים 3: מודל קליינט-שרת - מי יוזם



שלבים 2-3: תשאול (בקשה-תשובה). שלבים 5-6: דחיפה - הרדיו יוזם. לכן אפליקציית ניטור היא כמעט תמיד גם קליינט וגם מאזין בו-זמנית, ובדרך כלל בנויה על שני חוטים או על asyncio.

```

+-----+
| Monitoring App (GUI / logic) | <-- you test here
+-----+
| Management protocol (JSON / XML / SNMP) | <-- message format
+-----+
| TCP or UDP sockets | <-- ordinary networking
+-----+
| IP |
+-----+ <-- the radio's edge
| MANET routing (multi-hop, self-healing) | <-- invisible to the app
+-----+
| MAC / TDMA (who transmits when) |
+-----+
| SDR waveform (modulation, hopping) |
+-----+
| RF front-end + antenna |
+-----+

```

כל מה שמעל הקו הכפול הוא תחום האחריות שלך. מתחתיו – הרדיו.

תרשים 5: הזרימה המלאה כשצומת נופל

```

t=0s    Network is stable
        [A]---[B]---[C]          App: A ok, B ok, C ok

t=5s    B moves away, link degrades
        [A]- ? -[B]---[C]        RSSI for B: -62 -> -78 -> -91

t=7s    Link breaks
        [A]      [B]---[C]        Radio A: B is unreachable

t=7s    MANET routing recalculates: another path to C? -> no
        Radio A marks B and C as lost

t=8s    RADIO --"node B lost, node C unreachable"--> APP

t=8s    App updates display: B Disconnected | C Unreachable | A-B Down

        <-- convergence time = 7s to 8s = the number you measure

```

תרשים 6: איפה הבאגים מתחבאים

REALITY (the field)	RADIO'S VIEW (routing table)	WIRE (messages)	APP DISPLAY (what operator sees)
↓ gap 1	↓ gap 2	↓ gap 3	↓
gap 1: radio hasn't noticed the change yet	gap 2: event lost / never sent	gap 3: app received but rendered wrong	
			-> YOUR bug to find

התפקיד שלך הוא בעיקר פער 3, ולוודא שפער 2 לא משאיר את התצוגה שגויה לנצח: לגרום לאובדן אירוע בכוונה ולבדוק שהתשואול התקופתי מתקן.

תרשים 7: סביבת בדיקות בלי חומרה

```

graph TD
    A[pytest test case  
controls both sides] --> B[Your App  
under test]
    A --> C[Fake radio  
(Python UDP/TCP  
server you  
write yourself)]
    B <--> C
    C --> D[you decide what it reports]
    D --> E[scripted scenarios:  
- node joins / node lost  
- RSSI drops slowly  
- event dropped on purpose  
- delayed response (timeout)  
- malformed message]

```

מושגים מהפרק

ממשק ניהול – Management Interface

הערץ שדרכו האפליקציה מקבלת נתוני מצב.

צומת מקומי – Local Node

הרדיו שמחובר לאפליקציה בכבל – החלון היחיד שלה לרשת.

סנכרון מחדש – Resync

תשאול תקופתי שמתקן את המצב אם אירוע אבד.

מנוי לאירועים – Subscription

בקשה חד-פעמית לקבל עדכונים יוזמים מהרדיו.

מדמה – Simulator / Stub

רכיב מזויף שמחליף את הרדיו בבדיקות ומאפשר שליטה מלאה בתרחיש.

9 ארגז הכלים של מפתחת בדיקות

שורה תחתונה: הכלים הם חצי מהסיפור. מה שמפריד בודקת מתחילה ממפתחת בדיקות מקצועית הוא הארכיטקטורה (פרק 10).

שכבה	כלים	למה
מסגרות בדיקה	pytest, Robot Framework, NUnit/xUnit	תוספים ל-xdist: pytest (מקביליות), timeout (בדיקת רשת תקועה), repeat (יציבות), html/Allure (דוחות). Robot נפוץ לבדיקות מערכת; NUnit בצד #C.
ניתוח תעבורה	Wireshark, tshark, scapy	Wireshark מוכיח אם הבעיה באפליקציה או ברדיו. tshark ניתן לאוטומציה. scapy שולח חבילות פגומות בכוונה – בדיקות חוסן.
עומס והשפעה על רשת	iperf3, tc netem	iperf3 מודד תפוקה אמיתית. netem מזריק השהיה ואיבוד חבילות – הכלי החשוב ביותר כשאין חומרה.
מדמים וכפילים	unittest.mock, Radio Stub, pytest-httpserver, testcontainers	מדמה רדיו משלך הוא הכלי המרכזי בשנה הראשונה – עובד דרך הרשת האמיתית ולכן בודק גם את שכבת הסוקטים.
ציוד מדידה	PyVISA, SCPI, pySerial	שליטה במנתח ספקטרום, מחולל אותות, ספק כוח, מנחת. pySerial ל-UART.
תשתית CI	Git, Jenkins, Docker	הרצה אוטומטית על כל שינוי וסביבה זהה לכולם.
נתונים ודוחות	pandas, SQL, Grafana	ניתוח מגמות, זיהוי בדיקות רועדות, שאילתות היסטוריות.

דוגמאות שימוש

לכידה עם tshark בזמן בדיקה

```
# Capture management traffic during a test, then list event sizes
tshark -i eth0 -f "udp port 5000" -a duration:30 -w capture.pcap
tshark -r capture.pcap -T fields -e frame.time_relative -e data.len
```

הדמיית קישור טקטי גרוע עם netem

```
# Simulate a degraded tactical link: 200ms delay, 50ms jitter, 5% loss
sudo tc qdisc add dev eth0 root netem delay 200ms 50ms loss 5%

# Remove it
sudo tc qdisc del dev eth0 root
```

שליטה במכשיר מדידה עם PyVISA

```
import pyvisa

# Connect to an instrument and read its identity
rm = pyvisa.ResourceManager()
instrument = rm.open_resource("TCPIP::192.168.1.50::INSTR")
print(instrument.query("*IDN?"))

# Set a programmable attenuator to 30 dB to simulate distance
instrument.write("ATT 30")
```

ציוד מעבדת RF

קופסת מיגון – RF Shield Box

מבודדת את הרדיו מהסביבה כדי שהבדיקה תהיה חוזרת.

מנחת מתכנת – Programmable Attenuator

מדמה מרחק בין רדיואים. אוטומציה שמשנה הנחתה ובודקת תגובה – תרחיש קלאסי.

מדמה ערוץ – Channel Emulator
מדמה דעיכה, ריבוי נתיבים ותזוזה.

מנתח ספקטרום ומחולל אותות – Spectrum Analyzer / Signal
Generator
נשלטים מפייתון דרך PyVISA ו-SCPI.

תרגול בלי חומרה

מה נותן	כלי
רשת "גרועה" על כל מחשב לינוקס	tc netem
שרשרת SDR שלמה בתוכנה	GNU Radio
סימולציית MANET עם ניתוב ותזוזה	ns-3 / Mininet-WiFi
דונגל USB זול לקליטה בלבד – ספקטרום אמיתי	RTL-SDR

10 ארכיטקטורה של מסגרת בדיקות

Layered Architecture – הפרדת שכבות

+-----+ Test Layer (tests/) "when node B is lost, app shows Down" +-----+	<- business language only
+-----+ Service Layer (services/) wait_for_node_state(), get_topology() +-----+	<- domain actions
+-----+ Driver Layer (drivers/) socket, JSON parsing, retries +-----+	<- protocol details
+-----+ Infrastructure (config, logging) +-----+	

הכלל: בדיקה לעולם לא פותחת סוקט. אם הפרוטוקול משתנה מ-JSON ל-Protobuf, נוגעים רק בשכבת ה-Driver, וכל הבדיקות ממשיכות לעבוד.

SUT Abstraction – הפשטת המערכת הנבדקת

אותו קוד בדיקה רץ מול מדמה ומול חומרה, בעזרת ממשק אחיד ושני מימושים:

```

from abc import ABC, abstractmethod

class RadioDriver(ABC):
    """Common interface for real and simulated radios."""

    @abstractmethod
    def get_topology(self) -> dict:
        """Return the current network topology as reported by the radio."""

    @abstractmethod
    def get_link_quality(self, node_id: str) -> dict:
        """Return RSSI and SNR for a directly reachable neighbour."""

class SimulatedRadioDriver(RadioDriver):
    """Driver backed by a scripted scenario - used when no lab is available."""

    def __init__(self, scenario) -> None:
        self._scenario = scenario

    def get_topology(self) -> dict:
        return self._scenario.current_topology()

    def get_link_quality(self, node_id: str) -> dict:
        return self._scenario.current_link(node_id)

class RealRadioDriver(RadioDriver):
    """Driver talking to actual hardware over the management interface."""

    def __init__(self, host: str, port: int = 5000) -> None:
        self._host = host
        self._port = port

    def get_topology(self) -> dict:
        return self._send_request("GET_TOPOLOGY")

    def get_link_quality(self, node_id: str) -> dict:
        return self._send_request(f"GET_LINK {node_id}")

    def _send_request(self, command: str) -> dict:
        raise NotImplementedError("Protocol details come from internal docs")

```

fixture אחת בוחרת מימוש לפי אפשרות שורת פקודה:

```

import pytest

def pytest_addoption(parser):
    """Register the --target option (place this in conftest.py)."""
    parser.addoption("--target", default="sim", choices=["sim", "hardware"])

@pytest.fixture(scope="session")
def radio(request) -> RadioDriver:
    """Provide a radio driver based on the --target command line option."""
    if request.config.getoption("--target") == "hardware":
        return RealRadioDriver(host="192.168.1.10")
    return SimulatedRadioDriver(scenario=DefaultScenario())

```

זה מה שמאפשר להתחיל לכתוב בדיקות היום, בלי מעבדה, ולחבר לחומרה כשתגיעי.

עקרונות נוספים

המתנה חכמה – Explicit Wait

הטעות מספר אחת בבדיקות רשת: `time.sleep(5)` – גם איטי וגם לא אמין. במקום זה:

ניהול תצורה – Configuration Management

כתובות, פורטים וזמני המתנה – בקובץ YAML/TOML חיצוני, לא בקוד. סביבות נפרדות: `dev`, `lab`, `production`.

```
import time
from typing import Callable

def wait_until(
    condition: Callable[[], bool],
    timeout: float = 10.0,
    interval: float = 0.2,
) -> bool:
    """Poll a condition until it holds or the timeout expires."""
    deadline = time.monotonic() + timeout
    while time.monotonic() < deadline:
        if condition():
            return True
        time.sleep(interval)
    return False
```

Test Independence – עצמאות בדיקות

כל בדיקה מקימה את המצב שלה ומנקה אחריה. בלי זה אי אפשר להריץ במקביל.

Flaky Tests – בדיקות רועדות

ברשת אלחוטית כשלים אקראיים הם חלק מהחיים, אבל בדיקה רועדת היא רעל – אנשים מפסיקים להאמין לדוחות. כל כשל חייב הסבר: או שהמוצר תקול, או שהבדיקה לא נכונה.

Traceability – עקיבות

בתעשייה ביטחונית כל בדיקה ממופה לדרישה:

Test Pyramid – פירמידת בדיקות

הרבה בדיקות יחידה, פחות אינטגרציה, מעט בדיקות מערכת מלאות. רוב הבדיקות רצות מול מדמה.

```
@pytest.mark.requirement("REQ-1042")
def test_node_loss_is_reflected_in_ui(radio, app):
    """Verify the UI marks a node as Down after the radio reports its loss."""
```

11 רשתות לעומק – מה צריך לדעת כדי לבדוק

11.1 מחסנית הפרוטוקולים – OSI / TCP-IP

לא לשנן שבע שכבות, אלא לדעת לאבחן איפה בעיה יושבת:

סימפטום	שכבה
כבל מנותק	פיזית
כתובת MAC שגויה	קו נתונים
ניתוב שבור	רשת
פורט חסום	תעבורה
JSON פגום	אפליקציה

11.2 UDP לעומק

רשתות טקטיות משתמשות ב-UDP כי עדיף לאבד מסגרת וידאו מאשר לחכות לשידור חוזר. חבילות יכולות ללכת לאיבוד בשקט, להגיע כפולות או בסדר שגוי.

Multicast – שידור לקבוצה

טווח 224.0.0.0 – 239.255.255.255. חבילה אחת מגיעה לכמה מנויים. נפוץ בדיווחי מצב ברשתות טקטיות.

MTU – מגבלת גודל

בדרך כלל 1500 בתים באתרנט, ברדיו בדרך כלל פחות. הודעה גדולה מתפצלת, וב-UDP אובדן רסיס אחד מאבד את כל ההודעה. מסביר למה הודעות גדולות "נעלמות" והקטנות עוברות – תרחיש בדיקה מצוין.

11.3 TCP לעומק

לחיצת יד – Three-way Handshake

SYN, SYN-ACK, ACK. אם האפליקציה נתקעת ב-connect, ב-Wireshark רואים SYN בלי תשובה – הבעיה ברשת, לא בקוד.

שידור חוזר – Retransmission

TCP משדר שוב כשלא קיבל ACK. בהשגחה גבוהה זה מוריד תפוקה מאוד.

מצב TIME_WAIT

סוקט שנסגר נשאר תפוס כ-60 שניות – הסיבה הנפוצה ל-"Address already in use" בבדיקות רצופות. פתרון: SO_REUSEADDR או פורט דינמי.

אלגוריתם Nagle

TCP מאגד הודעות קטנות ומוסיף השהיה. מכבים עם TCP_NODELAY במערכות רגישות לזמן.

11.4 תשתית ושירותים

כתובות ותת-רשתות – IP Addressing / Subnetting

CIDR, מסכות רשת, פרטי מול ציבורי. חובה כדי להבין למה שני מכשירים לא רואים זה את זה.

פרוטוקול גילוי כתובות – ARP

מיפוי IP ל-MAC. `arp -a` – הפקודה הראשונה כשאינ תקשורת באותה רשת.

ניתוב – Routing

טבלת ניתוב, שער ברירת מחדל, `ip route`. ב-MANET הטבלה משתנה דינמית.

תעדוף תעבורה – QoS

סימון חבילות כך שקול יקבל עדיפות על קבצים. חיוני ברוחב פס מוגבל.

רשת וירטואלית – VLAN

הפרדת רשתות על אותו כבל. נפוץ במעבדות.

11.5 מושגי ביצועים

תפוקה – Throughput

כמה נתונים באמת עוברים, להבדיל מהקצב התיאורטי.

השהיה – Latency

זמן הלוח ושוב. נמדד ב-ping.

רעידת השהיה – Jitter

שונות בהשהיה. היא שהורסת שיחות קול, לא ההשהיה עצמה.

מכפלת רוחב פס-השהיה – Bandwidth-Delay Product

$BDP = Bandwidth \times RTT$ – כמה נתונים "בתעופה" בקו בכל רגע. מסביר למה TCP איטי בקווים ארוכים עם חוצץ קטן.

11.6 אבטחה

הצפנה ואימות – TLS / mTLS

הצפנה ואימות דו-כיווני בין קליינט לשרת.

סימטריה מול אסימטריה – AES vs RSA

AES להצפנת הנתונים עצמם, RSA להחלפת מפתחות וחתימות.

מודל איומים – Threat Model

Replay, Man-in-the-middle, הזרקת נתונים. באפליקציית ניטור: מה קורה אם מישו מזריק דיווח טופולוגיה מזויף?

11.7 כלי אבחון בלינוקס

<code>ip addr</code>	# interfaces and addresses
<code>ip route</code>	# routing table
<code>ss -tulnp</code>	# open sockets and owning processes
<code>ping -c 4 192.168.1.10</code>	# basic reachability
<code>traceroute 192.168.1.10</code>	# path and where it breaks
<code>tcpdump -i eth0 udp port 5000 -nn</code>	# quick capture without a GUI
<code>nc -u -l 5000</code>	# ad-hoc UDP listener for manual checks

ss החליף את netstat, ו-ip החליף את ifconfig.

תרחישי בדיקה לאפליקציית ניטור MANET

תרחיש	מה בודקים
צומת עוזב (Node lost)	האם הסטטוס משתנה, תוך כמה זמן, והאם הקישורים שלו מסומנים Down.
צומת מצטרף (Join)	האם מופיע, עם הנתונים הנכונים, בלי כפילויות.
פיצול ואיחוד (Partition / Merge)	האם הנתונים בממשק נשארים עקביים אחרי האיחוד.
איכות קישור יורדת	RSSI/SNR מוצגים נכון, ביחידות נכונות, וההתראה קופצת בסף הנכון.
RSSI לצומת מרוחק	האפליקציה לא מציגה RSSI לצומת שאינו שכן ישיר.
קישור אי-סימטרי	איזה כיוון מוצג, והאם זה מתועד.
אובדן אירוע	התשאול התקופתי מתקן את התצוגה.
השהיה גבוהה / timeout	הממשק לא קופא, מציג "לא עדכני" במקום נתון שגוי.
עומס במישור הנתונים	דיווחי הניהול עדיין מגיעים בזמן.
הודעה פגומה	האפליקציה לא קורסת ורושמת ללוג.
הודעה גדולה מה-MTU	מה קורה כשדיווח טופולוגיה גדול מתפצל ורסיס אובד.
דיווח מזויף	האם יש אימות מקור.

סדר עדיפויות ללמידה

1. Wireshark ו-tc netem – יכולת מיידית, בלי חומרה.
2. מדמה רדיו משלך – מלמד פרוטוקולים, סוקטים וארכיטקטורה בבת אחת.
3. ארכיטקטורות שכבות עם pytest – זה מה שיראו בקוד שלך.
4. scapy ובדיקות חוסן.
5. PyVISA ו-SCPI – כשתהיה גישה למכשירים.

פרויקט מומלץ: מדמה רדיו MANET בפיתוח שמדווה טופולוגיה ו-RSSI דרך UDP, אפליקציית ניטור קליינט מעליו, ארכיטקטורת שלוש שכבות, ובדיקות pytest לאובדן צומת, התכנסות ואירוע שאבד.

מילון מושגים מקוצר

מונח	בקצרה
RF	תקשורת בגלי רדיו
SDR	רדיו שממומש בעיקר בתוכנה
Waveform	חבילת התוכנה שמגדירה איך הרדיו משדר
Ad-hoc	רשת בלי תשתית מרכזית
MANET	רשת אד-הוק עם צמתים ניידים
Node / Link	צומת / קישור בין שני צמתים
Topology	מי מחובר למי כרגע
Multi-hop	מידע עובר דרך צמתי ביניים
Self-healing	מציאת מסלול חלופי אוטומטית
Merge / Join	איחוד תת-רשתות / צירוף צומת
RSSI / SNR	עוצמת אות נקלט / יחס אות לרעש
dB / dBm	יחס לוגריתמי / הספק ביחס ל-1 mW
Data / Management Plane	התעבורה עצמה / מידע על הרשת
Polling / Push	האפליקציה שואלת / הרדיו מודיע
Convergence Time	זמן התייצבות אחרי שינוי
State Consistency	התצוגה תואמת למציאות
Stub / Simulator	רדיו מזויף לבדיקות
MTU	גודל חבילה מרבי
Jitter	שונות בהשהיה

- rafael.co.il – BNET-HH datasheet
- Shephard Media – BNET SDR (Jan 2026), BNET demo to Romanian special forces (Feb 2026)
- Army Recognition – DX Korea 2018
- Astra Rafael Comsys – SW-define-radio

חומר פנימי ומסווג מהעבודה אינו חלק מהמסמך הזה ולא צריך להיכנס אליו.

13 ICD ופענוח פרוטוקולים בינאריים

שורה תחתונה: זה הפער החשוב ביותר בכל הרשימה. רוב באגי האינטגרציה אינם באגי רדיו אלא אי-הסכמה על החזרה בין שני הצדדים: סדר בתים הפוך, ביט אחד בסטייה, יחידה שונה, שדה אופציונלי לא מטופל.

מפרט דרישות ממשק – IRS

מסמך הדרישות שממנו ה-ICD נגזר. בדיקות הממשק מתחקרות אליו.

מסמך בקרת ממשק – ICD (Interface Control Document)

מסמך שמגדיר ומבקר את המידע העובר בין שתי מערכות או רכיבים – פריטי נתונים, הודעות, פרוטוקולים, ותזמון ורצף האירועים. זהו החוזה בין האפליקציה לרדיו.

מה ICD טוב מכיל

סוגי הודעות ומזהים, מבנה הכותרת, שמות שדות, טיפוסים, יחידות, טווחים חוקיים, סדר בתים, תעבורה (TCP/UDP ופורט), תזמון ורצף, קודי שגיאה וכללי גרסאות.

איך הופכים ICD לבדיקות – שישה שלבים

1. **קטלוג הודעות:** לכל הודעה – מזהה, כיוון, טריגר (בקשה/תשובה/אירוע/מחזורי), קצב וגודל.
2. **לכל שדה:** מינימום, מקסימום, ערך מחוץ לטווח, ערכים שמורים, אפס, ערך "לא ידוע", כל ערך ב-enum, כל ביט דגל.
3. **בדיקות שליליות לכל הודעה:** הודעה קטועה, בתים עודפים בסוף, שדה אורך שגוי, CRC פגום, מזהה הודעה לא מוכר, גרסה שגויה, הודעה כפולה או מחוץ לסדר.
4. **בדיקות רצף:** בקשה ללא תשובה (timeout), תשובה למזהה בקשה לא מוכר, אירוע שמגיע לפני שהחיבור הושלם, מטח אירועים.
5. **עקיבות:** לתייג כל בדיקה במזהה ההודעה או בסעיף ה-ICD, כדי שעדכון ICD יראה מיד אילו בדיקות להחליף.
6. **שיטות אימות:** בדיקה, ניתוח, הדגמה וניסוי – Inspection, Analysis, Demonstration, Test (IADT). כל דרישה במסמך מסומנת באחת מהן.

מושגי פרוטוקול בינארי

מסגור – Framing

TCP הוא זרם בתים, ולכן צריך להגדיר גבולות הודעה: **קידומת אורך** (הנפוץ ב-ICD בינארי), **תוחם** (תו מפריד עם byte stuffing), או **גודל קבוע**. בדיקה קלאסית: לפצל הודעה אחת לשתי קריאות recv, ולדחוס שתי הודעות לקריאה אחת. הרבה באגי לקוח מתגלים רק שם.

סדר בתים – Endianness

סדר בתים ברשת הוא big-endian, ומעבדי x86 הם little-endian. בפיתוח: > או ! ל-big, < ל-little.

שדות ביט – Bit Fields

הודעות צבאיות רבות אורזות שדות שאינם מיושרים לבייט (למשל 3 ביטים גרסה ו-5 ביטים דגלים). הודעות J-series של Link 16 הן מבנים ארוזים באורך קבוע.

סכום ביקורת – Checksum / CRC

בדיקת שלמות ההודעה. תרחיש בדיקה: לשנות ביט אחד ולוודא שההודעה נדחית ונרשמת ללוג.

קנה מידה קבוע – Fixed-point Scaling

קואורדינטות נשמרות לעיתים קרובות כ-int32 כפול 1e-7. טעות בקנה המידה מזיזה יעד בעשרות קילומטרים.

פורמטי סריאליזציה לזהות

פורמט	מאפיין
Protobuf	סכמה + varints, קומפקטי
ASN.1 (BER/DER/PER)	משמש ב-SNMP וב-X.509
JSON + JSON Schema	אימות מול סכמה הוא בדיקה זולה וחזקה
XML + XSD	הפורמט של CoT

struct למבנים פשוטים; Construct – ספרייה הצהרתית ברמת ביט וביט, שאותה הגדרה גם מפענחת וגם בונה, ולכן מושלמת למדמה רדיו שצריך לייצר הודעות תקינות ופגומות; bitstring לקריאה ברמת ביט; Kaitai Struct (הגדרה אחת, פרסרים בפייתון וב-C#); asn1tools; #C: BinaryPrimitives עם סדר בתים מפורש, <byte> Span ו-System.IO.Pipelines.

פרויקט מומלץ: לכתוב **Wireshark Lua dissector** לפרוטוקול שבין האפליקציה לרדיו, ישירות מה-ICD. זה גם לימוד מעולה וגם כלי שהצוות ישתמש בו.

14 מישור הניהול: syslog, SNMP ומצבי רדיו

שורה תחתונה: הפרק על "תשאול מול דחיפה" מקבל כאן פרוטוקול אמיתי. גם המצבים של הרדיו עצמו (תצורה, מפתחות, איפוס) משנים מה האפליקציה אמורה להציג.

SNMP – מה שבודקת צריכה לדעת

פעולות – Operations GET, GETNEXT/GETBULK (סריקה), SET, ו-TRAP/INFORM – הודעות יזומות מהסוכן. בדרך כלל 161 UDP לשאילתות ו-162 ל-traps.	מנהל וסוכן – Manager / Agent האפליקציה היא המנהל, ועל הרדיו רץ סוכן. הנתונים מתוארים ב-MIB, ולכל אובייקט יש OID.
הודעת מלכודת – Trap SNMP נשען בעיקר על תשאול, וה-trap רק מתריע למנהל שכדאי לתשאל שוב. traps אינן מאושרות (בניגוד ל-INFORM) ולכן אינן תחליף לתשאול. זו בדיוק בדיקת "אירוע אבד" – האם התשאול מתקן? מהפרק על עקביות מצב.	גרסאות – Versions v1/v2c משתמשות במחרוזת community בלי אבטחה אמיתית. v3 מוסיפה אימות והצפנה, וזו הגרסה שמערכת ביטחונית צריכה.
	MIB פרטי – Enterprise MIB יצרנים מגדירים MIB משלהם. מנהל שלא טען את ה-MIB לא יודע לפרש את ה-trap. בדיקה: גרסת קושחה חדשה מוסיפה trap – האם האפליקציה מתעלמת בחן או קורסת?

פרוטוקולי ניהול נוספים

ניהול תצורה מודרני – NETCONF / YANG / RESTCONF YANG מדגם את הנתונים, NETCONF (מעל SSH) או RESTCONF (מעל HTTP) מבצעים את השינוי, ו-telemetry דוחף מדדים. מחליף את SNMP במערכות חדשות.	יומן מערכת – Syslog רדיואים ושרתים שולחים לוגים ל-514 UDP/TCP, בפורמט מובנה לפי RFC 5424 עם דרגת חומרה 0-7. אוספים אותם מרכזית כדי להצליב עם לוגי האפליקציה.
--	--

מצבי תצורה ומפתחות (רמה עקרונית בלבד)

טעינת מפתחות – Key Fill מפתחות הצפנה נטענים באמצעות התקן טעינה ייעודי. בעולם האמריקאי הפרוטוקולים הם DS-101 ו-DS-102, כאשר DS-101 יכול לטעון גם אלגוריתמים ועדכוני תוכנה.	תצורה מוגדרת מראש – Preset / Mission Plan הרדיו נטען בתצורה: תדרים, מזהי רשת, הספק, תפקיד. השלכה לבדיקה: האפליקציה צריכה להציג איזו תצורה פעילה, ולטפל בצומת שמצטרף עם תצורה שונה.
איפוס מפתחות – Zeroize מחיקת כל חומר ההצפנה. מסמך מדיניות אבטחה פומבי של התקן ניהול מפתחות טקטי מציין שאחרי zeroize יש לטעון דינית לפחות KEK אחד לפני שהיחידה יכולה להשתתף שוב ב-OTAR.	החלפת מפתחות באוויר – OTAR עדכון מפתחות דרך הרשת עצמה, אבל מפתח-אב (KEK) בדרך כלל עדיין נטען פיזית.

למה זה חשוב לבודקת: צומת מאופס או עם מפתח שגוי הוא "נוכח אבל לא מדבר". האפליקציה חייבת להבחין בין שלושה מצבים שונים: **צומת נפל, צומת פעיל אך לא ברשת ההצפנה, וצומת פעיל עם תצורה שונה.** לכל אחד תצוגה והתראה אחרות. לעולם לא לעבוד עם חומר מפתח אמיתי בסביבת בדיקות – רק מפתחות בדיקה לפי נוהלי הביטחון.

תרגול בבית

Net-SNMP (snmpwalk, snmptrapd), pysnmp לכתובת מנהל וסוכן, snmpsim שמדמה סוכן מתוך סריקה מוקלטת – מדמה מצוין למישור הניהול של הרדיו, ו-rsyslog.

שורה תחתונה: בלי שעון משותף אי אפשר למדוד זמן התכנסות. "צומת נפל ב-12:00:03" ברדיו מול "האפליקציה הראתה אדום ב-12:00:05" במחשב – אם השעונים לא מסונכרנים, המדידה חסרת ערך.

פרוטוקול זמן רשת – NTP

חותמות זמן בתוכנה מעל IP. לפי סיכום פרויקט NTP של דיוויד מילס, הדיוק הוא בסדר גודל של 100 מיקרו-שניות ברשת אתרנט 100 מגה-ביט בעומס נמוך, ועד עשרות מילישניות במסלול בין-יבשתי. מספיק להצלבת לוגים ברמת המילישנייה.

פרוטוקול זמן מדויק – PTP / IEEE 1588

חותמות זמן בחומרה נותנות דיוק תת-מיקרו-שנייה, אבל מניחות שההשהיה סימטרית בשני הכיוונים. תורים במתגים רגילים שוברים את ההנחה הזו ופוגעים בדיוק, אלא אם המתג משמש כשעון שקוף או גבולי.

למה זה חשוב ברדיו

שיטות גישה מסוג TDMA מחלקות את הזמן לחריצים, ולכן כל הצמתים זקוקים לבסיס זמן משותף. הודעת רפאל מאוקטובר 2021 על BNET NANO מציינת שהרדיו פועל באזורים ללא GPS, כלומר הוא שומר על תזמון רשת ללא GPS. כבודקת ייתכן שתדרשי לאמת התנהגות כאשר זמן GPS אובד, חוזר, או קופץ.

למה זה חשוב לאפליקציה

- **מתאם לוגים:** מדידת זמן התכנסות דורשת ששני השעונים יסכימו.
- **לוגיקת התיישנות:** אם האפליקציה מיישנת נתונים לפי חותמת זמן, סטיית שעון יוצרת צמתים "מיושנים" מזויפים או כאלה שלעולם אינם מתיישנים. לבדוק עם סטייה מכוונת של שניות, דקות, וקפיצה אחורה.
- **שעון מונוטוני:** למדידת משכים בבדיקות תמיד `time.monotonic()` בפיתוח או `Stopwatch` ב-C#, לעולם לא שעון קיר ש-NTP יכול לקפוץ בו.

לוגים מובנים – Structured Logging

לרשום ב-JSON עם השדות `ts` (UTC, ISO-8601 עם מילישניות), `node_id`, `component`, `event`, ו-`seq` או `correlation_id`. אפשר למזג צירי זמן בין צמתים עם `pandas` או `Grafana Loki`. **תרגול:** `chrony` (בדיקה עם `chronyc tracking`), `linuxptp` ו-`libfaketime` להזרקת סטיית שעון לתהליך.

16 בניית רשת רב-צמתית על מחשב אחד

שורה תחתונה: אפשר להריץ עשרות "רדיואים" על מחשב אחד בעזרת מרחבי שמות של לינוקס. זו הדרך המעשית לבדוק פיצול ואיחוד רשת בלי חומרה.

אבני הבניין בלינוקס

מרחב שמות רשת – Network Namespace

`ip netns add n1` יוצר מחסנית רשת מבודדת. כל מרחב מתנהג כמו "מחשב רדיו" נפרד.

זוג veth Pair – veth

כבל וירטואלי עם שני קצוות. קצה אחד בכל מרחב שמות.

גשר – Linux Bridge

מתג וירטואלי שמחבר הרבה veth, ויוצר "אוויר" משותף.

מוסיפים `tc netem` על כל veth כדי לתת לכל קישור איבוד והשהיה משלו, ומשנים בזמן ריצה כדי לדמות צומת שמתרחק. להנחתה אי-סימטרית: `netem` בכיוון אחד בלבד.

```
# Build a two-node topology on one machine
sudo ip netns add n1
sudo ip netns add n2
sudo ip link add veth1 type veth peer name veth2
sudo ip link set veth1 netns n1
sudo ip link set veth2 netns n2
sudo ip netns exec n1 ip addr add 10.0.0.1/24 dev veth1
sudo ip netns exec n2 ip addr add 10.0.0.2/24 dev veth2
sudo ip netns exec n1 ip link set veth1 up
sudo ip netns exec n2 ip link set veth2 up

# Degrade only the n1 -> n2 direction (asymmetric link)
sudo ip netns exec n1 tc qdisc add dev veth1 root netem loss 20% delay 150ms
```

אמולטורים שמשמשים בפועל למחקר MANET טקטי

אמולטור מחקר פתוח – CORE

בונה צמתים וירטואליים כמרחבי שמות של לינוקס, עם ממשק גרפי וממשק פייתון. כברירת מחדל משתמש בגישור וב-netem. בשילוב עם EMANE, CORE מדמה את שכבה 3 ומעלה ו-EMANE את שכבות 1-2, דרך התקן TAP בכל מרחב שמות.

אמולטור רשת אד-הוק ניידת – EMANE

פותח במעבדת המחקר הימית האמריקאית (NRL). הוא מבצע מידול בזמן אמת של הקישוריות בשכבת הקו והשכבה הפיזית, כך שתוכנת פרוטוקולים ואפליקציות אמיתית רצה מעליו. יש בו מודלים מתחלפים של PHY ו-MAC, כולל דעיכת מסלול, השפעות אנטנה והפרעות. סקירת ספרות מתארת אותו כאמולטור מוערך לרשתות טקטיות, ומציינת שחסר בו מודל הפרעות מדויק מבוסס SINR.

אמולטורים נוספים – Mininet-WiFi, ns-3, GNS3

Mininet-WiFi קל ונוח ומוסיף אלחוט וניידות, אך פחות ריאליסטי מבחינת רדיו. ns-3 הוא סימולטור אירועים בדידים, טוב לחקר ניתוב בקנה מידה ופחות לבדיקת בינארי אמיתי. GNS3 מתאים לגב הרשת הקווי.

המלצה מעשית: מרחבי שמות עם netem לבדיקות מהירות ב-CI, ו-CORE (ובהמשך עם EMANE) לתרחישי ניידות, פיצול ואיחוד ריאליסטיים. EMANE יכול לייבא מיקומי צמתים מקובצי ניידות, כך שאפשר לשחזר תרגיל שדה במעבדה.

17 תהליך הבדיקות בתעשייה הביטחונית

שורה תחתונה: הבדיקות האוטומטיות שלך יושבות בתוך מסגרת מסמכים פורמלית. כדאי לדעת את שמות המסמכים והשלבים כבר ביום הראשון.

MIL-STD-498 – האב של תבניות התיעוד

התקן פורסם ב-5 בדצמבר 1994 ובוטל ב-27 במאי 1998 לטובת IEEE/EIA 12207, אך תבניות המסמכים שלו עדיין בשימוש נרחב.

פריט תצורת תוכנה – CSCI

צבר תוכנה שממלא תפקיד שימושי מוגדר ומיועד לניהול תצורה נפרד מטעם הרוכש.

בדיקת הסמכה – Qualification Testing

בהגדרת התקן: בדיקות שנועדו להוכיח לרוכש ש-CSCI או מערכת עומדים בדרישות שהוגדרו להם.

כלל העצמאות – Independence Rule

התקן קובע שמי שאחראי על בדיקות ההסמכה של CSCI לא יהיה מי שביצע את התכן המפורט או המימוש שלו. זו הסיבה שקיים צוות בדיקות נפרד.

רמות הבדיקה בתקן

Unit testing → Unit integration & testing → CSCI qualification testing → CSCI/HWCI integration & testing → System qualification testing

שלושת מסמכי הבדיקה

מסמך	תוכן
STP – תוכנית בדיקות	סביבת הבדיקה, אילו בדיקות יבוצעו, ולוח הזמנים
STD – תיאור בדיקות	ההכנות, מקרי הבדיקה והנהלים. כאן חיות המפרטים של הבדיקות האוטומטיות
STR – דוח בדיקות	תיעוד בדיקות ההסמכה שבוצעו, כלומר התוצאות

בנוסף: SRS (מפרט דרישות תוכנה) ו-IRS (מפרט דרישות ממשק). זוג ה-IRS/ICD הוא היעד שאליו מתחקרות בדיקות הממשק.

בדיקות קבלה

בדיקת קבלה באתר – SAT

מבוצעת באתר ההתקנה ומראה שההתקנה תואמת לתקנים, למפרטים ולהוראות ההתקנה. SIT מראה ששילוב תת-המערכות עובד כמערכת אחת.

בדיקת קבלה במפעל – FAT

לפי IEC 62381: בדיקה שמדגימה שהמערכת או הרכיב תואמים למפרט, ומבוצעת בדרך כלל במתקן הספק. חבילת האוטומציה מורצת לא פעם כחלק ממנה.

אימות מול תיקוף – Verification vs Validation

לפי ניסוח ISO 9000, אימות הוא אישור באמצעות ראיות אובייקטיביות שדרישות מוגדרות מולאו, ותיקוף הוא אישור שהדרישות לשימוש המיועד הספציפי מולאו. הניסוח של בוהם: אימות שואל "האם אנחנו בונים את המוצר נכון", ותיקוף שואל "האם אנחנו בונים את המוצר הנכון".

משמעות מעשית: בדיקות ה-ICD והדרישות האוטומטיות שלך הן בעיקר אימות. תרגיל שדה שבו מפעילים שופטים אם תמונת הניטור שימושית הוא תיקוף.

מודל V ומטריצת עקיבות

כל מסמך בצד שמאל של ה-V מתאים לרמת בדיקה בצד ימין. **מטריצת העקיבות (RTM)** ממפה מזהה דרישה למזהה מקרה בדיקה ולתוצאה. כלים: DOORS, Jama, Polarion. בבית: marker ב-pytest וסקריפט שמייצא CSV.

למודעות בלבד: DO-178C (תוכנה מוטסת) מדרג רמות A עד E לפי חומרת הכשל – A מחייב כיסוי MC/DC, B כיסוי החלטות, C כיסוי משפטים. רלוונטי אם תוכנה מוטסת נכנסת לתחום. IEC 61508 הוא המקבילה התעשייתית עם רמות 1-4 SIL.

18 Multicast לעומק

שורה תחתונה: דיווחי מצב ומיקום נשלחים לעיתים קרובות ב-multicast, וזו הסיבה הנפוצה ביותר לבדיקה שעוברת על שולחן אחד ונכשלת על אחר.

האזנת מתג – IGMP Snooping

המתג מאזין לדיווחים ומעביר את הקבוצה רק ליציאות המעוניינות.

פרוטוקול ניהול חברות – IGMP / MLD

מארחים שולחים דיווח הצטרפות לקבוצה, ו**שואל** (בדרך כלל נתב) שואל מדי פעם מי עדיין מעוניין בקבוצה.

למה multicast "נעלם" במעבדה: בלי שואל ב-VLAN, רשומות ה-snooping פגות אחרי הדיווחים הראשונים. ההתנהגות משתנה לפי יצרן: מומחי קהילת Cisco מציינים שבמתגי Catalyst עם IOS ה-snooping מושבת והתעבורה מוצפת לכל מקום, ואילו במתגי Nexus, בלי שואל, ה-multicast פשוט לא יעבוד. התוצאה: הבדיקה עוברת על ספסל אחד ונכשלת על אחר, או עובדת כמה דקות ואז נעצרת. הפתרון: להפעיל שואל IGMP במתג, או לכבות snooping ב-VLAN הבדיקה.

מלכודות נוספות

- **TTL=1** כברירת מחדל ל-multicast – לא יחצה נתב.
 - **ממשק שגוי** במחשב עם שני כרטיסי רשת – תמיד להגדיר `IP_MULTICAST_IF` ולהצטרף על ממשק מפורש.
 - **חומת אש** מקומית שחוסמת 224.0.0.0/4.
 - **גשר Docker** שאינו מעביר multicast כברירת מחדל.
 - **נקודות גישה אלחוטיות** שממירות או מגבילות קצב multicast.
 - **בין נתבים** נדרש PIM. ב-MANET ה-multicast בדרך כלל מטופל בתוך הרדיו, ולכן צד האפליקציה צריך לצפות לאיבוד ולכפילויות.
- רעיונות לבדיקה:** האפליקציה מצטרפת לקבוצה והמדמה שולח; להרוג את השואל; להפעיל מחדש את האפליקציה (האם היא מצטרפת שוב?); שני כרטיסי רשת; חבילות כפולות משני מסלולי מסר.

19 סוקטים לעומק ודפוסי אמינות

סוקטים – שכבת התכנות

חוצצי סוקט – Socket Buffers

אפליקציה איטית יכולה לגרום לגלישת חוצץ הקליטה של UDP ולאיבוד אירועים בשקט. לבדוק `netstat -su` בזמן בדיקת עומס.

חוסם מול לא-חוסם – Blocking vs Non-blocking

תמיד להגדיר timeout על כל קריאת סוקט בקוד בדיקה. לריבוי חיבורים: `select` / `poll` / `epoll`, או `selectors` ו-asyncio בפיתון.

שמירת חיים – TCP Keepalive

TCP לבדו עלול לקחת זמן רב מאוד לזהות עמית שנעלם (ניתוק חשמל, שליפת כבל). לבדוק תרחיש "הרדיו נעלם בלי FIN ובלי RST".

שימוש חוזר בכתובת – SO_REUSEADDR / SO_REUSEPORT

הראשון מאפשר קשירה מחדש מיד אחרי הפעלה מחדש למרות TIME_WAIT. השני מאפשר כמה סוקטים על אותו פורט, מה שנדרש כששני תהליכי בדיקה מאזינים לאותה קבוצת multicast.

חיבור מחדש – Reconnect

השהיה מעריכית עם רעש אקראי. חשוב: לוודא שאחרי חיבור מחדש האפליקציה מסנכרנת מצב מלא, ולא ממשיכה מהדלתא.

סגירה מסודרת – Graceful Shutdown / Half-close

לוודא שהלקוח מבדיל בין FIN, RST ומוות שקט ומגיב לכל אחד נכון.

דפוסים אמינות – כל אחד הוא עדשת בדיקה

דפוס	מה לבדוק
מספרי סידור – Sequence Numbers	פערים, גלישה (0→65535), אתחול אחרי הפעלה מחדש של הרדיו
אידימפוטנטיות – Idempotency	פקודה חוזרת לא תפעל פעמיים
הסרת כפילויות – Deduplication	אותו אירוע מגיע משני מסלולים ברשת מש
ניסיונות חוזרים – Retries	מספר ניסיונות חסום, ותשובה שמגיעה אחרי ה-timeout
לחץ נגדי – Backpressure	אירועים מגיעים מהר יותר מהתצוגה – תור גדל, זיכרון, השהיה
פעמת לב – Heartbeat / Watchdog	"אין נתונים מהרדיו N שניות" הוא מצב שונה מ"הרדיו מדווח שהצומת נפל"
תצלום ודלתא – Snapshot + Delta	באג קלאסי: דלתא מגיעה לפני שהתצלום המלא הושלם

כלי הזרקת תקלות: toxiproxy מוסיף השהיה, הגבלת רוחב פס, timeout ואיפוס חיבור בין האפליקציה למדמה.

20 בדיקות ממושכות, הזרקת תקלות וקנה מידה

בדיקת עומס קיצון – Stress

הודעת רפאל על BNET NANO משווקת תמיכה ברשתות "שטוחות" עם מאות משתתפים, ולכן כדאי לבדוק את האפליקציה עם מאות צמתים מדומים וקצב אירועים מעבר לשיא הצפוי.

בדיקת סבולת – Soak / Endurance

להריץ תחלופה ריאליסטית (צמתים מצטרפים ועוזבים, איכות משתנה) במשך 8 עד 72 שעות, ולעקוב אחרי זיכרון, מספר ידיות וחוסים, מעבד, עומק תורים, היענות הממשק ונפח הלוגים. שיפוע עולה, לא קפיצה, מגלה דליפה.

הזרקת תקלות – Fault Injection / Chaos

להרוג את תהליך הרדיו המקומי, לנתק את קישור הניהול, לפצל את הרשת המדומה לשניים ולאחד, להזריק קפיצת שעות, לשבש בתיים, למלא את הדיסק של הלוגים, ולהפעיל מחדש את האפליקציה באמצע מוט.

בדיקת קיבולת – Capacity / Scalability

למדוד כיצד זמן רענון התצוגה, המעבד והזיכרון גדלים עם מספר הצמתים (50 → 100 → 200 → 500). מספר הקישורים גדל מהר יותר ממספר הצמתים.

כלים: memray ו-tracemalloc בפייתון; dotMemory, PerfView ו-dotnet-counters ב-C#; Performance Monitor של Windows לדליפות ידיות בממשק; Pumba לכאוס ב-Docker; ו-STOP - kill כדי לדמות רדיו תקוע.

בחירת גרסיה ודיווח

לתייג בדיקות (smoke / ICD / scale / soak), להריץ smoke בכל commit, מלא בלילה, ו-soak שבועי. הדיווח למנהלים: הצלחה וכישלון לפי דרישה, גרפי מגמה (זיכרון לאורך soak, זמן התכנסות לפי גרסה), תקלות פתוחות לפי חומרה, והצהרה ברורה על התצורה שנבדקה – גרסת אפליקציה, קושחה, טופולוגיה והגדרות האמולטור.

תוצאת בדיקה בלי תיעוד התצורה שבה איננה רצויה.

21 אוטומציית ממשק משתמש שולחני

שורה תחתונה: לאתר רכיבים לפי מזהה יציב ולא לפי קואורדינטות מסך. ואם אפשר, לבדוק את מודל הנתונים של האפליקציה ולא את הפיקסלים.

אוטומציית ממשק של מיקרוסופט – UI Automation (UIA)

כל פקד הוא רכיב בעץ. לאתר לפי AutomationId – יציב ובלתי תלוי בשפה. לבקש מהמפתחים להגדיר AutomationId לפקדים חשובים היא דרישת בדיקות לגיטימית.

כלי	מתי
FlaUI (#C)	רישיון MIT, עוטף UIA2 ו-UIA3 (בגסות: UIA3 ל-WPF, UIA2 ל-WinForms), בתחזוקה פעילה. גרסה 5.0.0 יצאה ב-25 בפברואר 2025 והוסיפה תמיכה ב-.NET 8
pywinauto (Python)	רישיון BSD, מנועי Win32 ו-UIA. משתלב ישירות עם pytest
Squish	מסחרי, התקן המקובל לאפליקציות Qt
Playwright	אם הממשק מבוסס דפדפן

לא להתחיל עם WinAppDriver: בדף הגרסאות ב-GitHub, הגרסה היציבה האחרונה v1.2.1 פורסמה ב-5 בנובמבר 2020, ובפרויקט יותר מ-1,100 תקלות פתוחות. תיעוד Appium מפנה היום ל-NovaWindows Driver כתחליף. Coded UI הוצא משימוש.

כלי בדיקה חזותיים: Accessibility Insights for Windows, Inspect.exe, FlaUIInspect.

מפה או תצוגת טופולוגיה: קנבס גרפי בדרך כלל לא חושף רכיב UIA לכל צומת. עדיף לאמת את **מודל הנתונים** של האפליקציה (דרך API, לוג או וו בדיקה), ולהשאיר לממשק רק שכבה דקה של בדיקות עשן והשוואות צילום מסך.

22 בדיקות מבוססות מודל ותכונות

אפליקציית ניטור היא אוסף מכונות מצבים: חיבור (מנותק → מתחבר → מסונכרן → מדרדר), צומת (לא ידוע → פעיל → מדרדר → אבוד → מיושן → הוסר), ורשת (אחת → מפוצלת → מאוחדת).

בדיקה מבוססת תכונות – Property-Based Testing

במקום דוגמאות בודדות, מגדירים תכונה שחייבת להתקיים תמיד, והכלי מייצר קלטים.

בדיקה מבוססת מודל – Model-Based Testing

לכתוב מודל ייחוס פשוט (למשל מילון פייתון של צמתים וקישורים צפויים), להזרים רצפי אירועים אקראיים גם למערכת הנבדקת וגם למודל, ולהשוות.

מכונת מצבים ב-Hypothesis – RuleBasedStateMachine

מגדירים **כללים** (פעולות כמו "צומת מצטרף", "קישור מדרדר", "החיבור נופל") ו**אינוריאנטות** ("אין צומת כפול בתצוגה", "לכל קישור מוצג יש שני קצוות קיימים"). Hypothesis מייצר רצפי פעולות, ובכישלון **מכווץ** אותם לרצף המינימלי שמשחזר את הבאג. זה בעל ערך עצום לבאגי פיצול ואיחוד.

לפרסרים: התכונה $build(parse(x)) == x$, והתכונה "הפרסר לעולם לא קורס על בתים אקראיים אלא זורק שגיאה מוגדרת" – צורה קלה של fuzzing. כלים נוספים: atheris, boofuzz לפרוטוקולי רשת. ב-C#: FsCheck, CsCheck.

23 נתוני תמונת מצב: CoT, PLI, ו-TAK

מעקב כוחות ידידותיים – Blue Force Tracking

זוויות בדיקה: קצב עדכון מול רוחב פס, טיפול במיקום מיושן, סימון "אין קיבוע" בהיעדר GPS, פורמטי קואורדינטות (WGS-84, MGRS), וייחוס גובה (מעל האליפסואיד או מעל פני הים).

מידע מיקום – PLI (Position Location Information)

דיווחי מיקום ומצב מחזוריים של כוחות. אחד המטענים המרכזיים שרדיו טקטי נושא ושאפליקציית ניטור מציגה.

סמן על מטרה – Cursor on Target (CoT)

פורמט אירוע ב-XML שנוצר ב-2002 על ידי MITRE ומרכז מערכות אלקטרוניות של חיל האוויר האמריקאי.

מבנה אירוע CoT

```
<event uid="..." type="a-f-G-U-C" how="..." time="..." start="..." stale="...">
  <point lat="32.05" lon="34.78" hae="35.0" ce="9.9" le="9.9"/>
  <detail> ... extensible application data ... </detail>
</event>
```

type hierarchy: a-f-G-U-C = atom / friendly / ground / unit / combat

ה-type הוא היררכי, ה-point כולל שגיאות מיקום (ce, le), וה-stale הוא זמן התפוגה המובנה – מודל מצוין לבדיקת לוגיקת התיישנות.

סביבת TAK

ATAK (אנדרואיד), iTAK, WinTAK, ו-TAK Server. ברירת המחדל לתמונת מצב מקומית היא UDP multicast בכתובת 239.2.3.1:6969. FreeTAKServer (פייתון, קוד פתוח; לקוחות מתחברים ב-TCP 8087 כברירת מחדל) ו-PyTAK חינמיים לתרגול בבית.

תרגיל משולב מצוין: FreeTAKServer + PyTAK + שני מרחבי שמות – לשלוח CoT ב-multicast, לשבור את זה בטעות IGMP או TTL או ממשק, לתקן, ואז לבדוק את לוגיקת התפוגה. זה מכסה multicast, אימות XML מול סכמה, התיישנות ו-asyncio בבת אחת.

פורמטים נוספים לזהות בשמם

פורמט	מה זה
NFFI	פורמט NATO למידע על כוחות ידידותיים
VMF	פורמט הודעות משתנה, מכון ביטים, לרשתות קרב צרות פס
JREAP	MIL-STD-3011, נושא הודעות Link 16 מעל IP ומדיה ארוכת טווח
SIMPLE	STANAG 5602

אם BNET או אפליקציית הניטור משתמשים בפורמטים האלה – זה מידע פנימי. כאן זה הקשר תעשייתי בלבד.

אלה בעיקר אוצר מילים לשיבות ולמסמכי דרישות. לא צריך שלוט בפרטים.

דוגמאות אמריקאיות

צורת גל לרדיו חייל – SRW (Soldier Radio Waveform)

צורת גל MANET לכוחות קטנים ולכלים בלתי מאוישים. מסמך IDA שבחן את ביצועיה בניסוי מבצעי דיווח על טווח ירוד בהשוואה למערכות ותיקות, צריכת הספק מופרזת, ורמה גבוהה של ניהול רשת וספקטרום שייכתן שאינה ישימה מבצעית. אלה בדיוק האתגרים שבדוקת נדרשת למדוד.

MANET מסחרי – TSM

צורת גל מסחרית שהצבא האמריקאי אימץ במקום SRW בשל קנה מידה וטווח.

נוספים: VULOS (קו ראייה V/UHF), SINCARS ו-HAVE QUICK (קפיצות תדר ותיקות לקול), ANW2.

אירופה ו-NATO

צורת גל בקצב גבוה – ESSOR HDRWF / STANAG 5651

אומצה על ידי NATO. פועלת ב-UHF בטווח 225–400 מגה-רץ, עד 1 מגה-ביט לשנייה, כוללת מצב MANET, ופועלת ללא לוויני ניווט.

צורת גל צרת פס – NBWF / STANAG 5630

מהדורה 2 צפויה להתחיל תהליך אשרור ברבעון האחרון של 2026. זו תחזית, לא עובדה מוגמרת.

נוספים: (SATURN) STANAG 4372 – UHF אווירי עמיד לשיבוש; STANAG 5066 ו-4538/4539 – נתונים ב-HF.

קישור נתונים טקטי – Link 16

TDMA עם קפיצות תדר בטווח 960–1215 מגה-רץ, הודעות J-series ארוזות ביטים, מוגדר ב-MIL-STD-6016 וב-STANAG 5516. הדוגמה הקנונית לרשת TDMA שדורשת ייחוס זמן רשת, ולבדיקת הודעות ארוזות ביטים.

חומר פומבי מציין שקיימת גרסת BNET שאושרה על ידי ה-FCC בתחומי LMR ומשתלבת עם מרכזיות דרך SIP. שווה להכיר יסודות: VoIP, SIP, RTP, מודד איכות קול MOS.

25

לוחמה אלקטרונית וניהול ספקטרום

סוגי שיבוש

סוג	מנגנון
נקודתי – Spot	כל ההספק על תדר אחד
סורק – Sweep	ההספק המלא נע על פני התדרים בתור
מטח – Barrage	רעש על פני רוחב פס גדול במחזור עבודה מלא, ודורש הספק רב
עוקב – Follower / Reactive	מזהה שידור ומשבש אותו. יעיל נגד קופצי תדר איטיים
מטעה – Deceptive / DRFM	מחזיר אות מזויף
מודע פרוטוקול	תוקף ערוצי בקרה או מסגרות ניהול

הסתברות יירוט וגילוי נמוכה – LPI / LPD

הודעת רפאל על BNET NANO מציינת שהמערכת מסוגלת לחוש הפרעות ספקטרום ולהימנע מהן, ופועלת כהתקן LPI/LPD בסביבות צפופות. אימות תכונות כאלה נעשה עם מחולל אותות או מדמה ערוץ שמזריק כל סוג שיבוש. תפקיד אפליקציית הניטור הוא **לדווח נכון**: ירידת איכות, החלפת תדר, אובדן צומת, וזמן התאוששות.

ניהול ספקטרום – Spectrum Management

הקצאת תדרים ורשתות ליחידות, ומניעת התנגשות בין רשתות ובין משדרים אחרים. רדיו קוגניטיבי מפחית תכנון ידני, אבל עדיין צריך לבדוק התנהגות כשהספקטרום המותר מצומצם.

הפרעה לא מכוונת – Co-site Interference

כמה רדיואים על אותו רכב מחלישים זה את רגישות זה. גם הפרעות אלקטרומגנטיות מהפלטפורמה עצמה.

תרגול בבית: להוסיף ל-flowgraph ב-GNU Radio משבשי טון, סריקה ורעש על תרגיל ה-QPSK מפרק 4, ולשרטט BER מול סוג שיבוש ומול יחס שיבוש לאות.

שורה תחתונה: בזמן בדיקות סביבה וחסיונות אלקטרומגנטית, התוכנה חייבת לרוץ ולהוכיח שהיא ממשיכה לעבוד. מהנדסות אוטומציה הן שכותבות את "תוכנת המאמץ" הזו.

תקן תאימות אלקטרומגנטית – MIL-STD-461G

משפחות הבדיקה: CE (פליטות מוליכות), CS (חסיונות מוליכה), RE (פליטות מוקרנות), RS (חסיונות מוקרנת) – למשל, RE102, CE102, RS103, CS114. קיימת מהדורה חדשה יותר, 461H, והתוכנית מפנה למהדורה שבחזרה.

תקן סביבה – MIL-STD-810H

פורסם ב-2019, עם הודעת שינוי 1 ב-2022. כולל שיטות בדיקה לטמפרטורה, לחות, הלם, רעידות, אבק, גשם וגובה, המותאמות לפרופיל הסביבתי של הציד. זהו תקן ולא תעודה – טענת "עומד ב-810" תלויה באילו שיטות ורמות נבחרו.

בדיקה עצמית מובנית – BIT / BITE

בדיקה בהדלקה, בדיקה רציפה, ובדיקה יזומה. לוודא שתקלות מזוהות ומדווחות לאפליקציית הניטור היא משימת אימות חומרה קלאסית.

מה כותבים: סקריפט שמפעיל את היחידה ורושם מדדים לאורך הבדיקה – תפוקה, מוני שגיאות ואובדן קישור – כדי להוכיח שהיחידה המשיכה לתפקד בזמן רעידות או הקרנה.

27 נושאים משלימים ותוכנית תרגול

נושאים משלימים

אבטחה בצד הלקוח – Client Security

אימות קלט לכל שדה ב-ICD (fuzzing), אימות החיבור לניהול, איסור סודות בלוגים, התנהגות בפקיעת תעודות, והרשאות מינימליות. מושגים ארכיטקטוניים להכיר: TEMPEST והפרדת אדום-שחור בין הצד הגלוי למוצפן.

ניהול תצורה של ספסל הבדיקות – Test Bench Configuration

לתעד גרסאות מדויקות של קושחה, אפליקציה ואמולטור בכל הרצה.

עדכון קושחה – Firmware Update / Rollback

עדכון תוך כדי פעילות, רשת עם גרסאות מעורבות (תאימות לאחור היא תכונה מוצהרת של BNET), והפסקת חשמל באמצע עדכון.

הספק וסוללה – SWaP

גודל, משקל והספק הם שיקול מרכזי ברדיו טקטי. דיווח מצב סוללה והתנהגות בסוללה חלשה הם תכונות ניטור אופייניות.

קול ותעדוף – Voice / PTT / QoS

השהיית לחיצה לדיבור, עדיפות קבוצות קול על נתונים, וחוצצי ריצוד. מאחר ש-MCR הוא תכונת דגל, צפויים תרחישים של כמה קבוצות קול במקביל עם נתונים ווידאו.

משמעת ניסוי שדה – Field Test Discipline

כרטיסי בדיקה, תיעוד מסלול GPS לצד הלוגים, ושחזור הלוגים באמולטור אחרי התרגיל.

תוכנית תרגול מוצעת, ארבעה שבועות

שבוע	מה עושים
1-2: ICD	לקחת ICD כלשהו (או סכמת CoT פומבית) ולכתוב: הגדרת Construct, בדיקת תכונה של הלוח ושוב, מחולל בדיקות שליליות, ו-Wireshark Lua dissector
2-3: מישור ניהול	להריץ snmpsim כסוכן רדיו מזויף, לכתוב תשאול ומקלט traps ב-pysnmp, ולבדוק "trap אבד – האם התשאול מתקן?"
3-4: טופולוגיה	סקריפט פייתון שמקים 10 מרחבי שמות, גשר ו-netem. תרחישים: עזיבת צומת, קישור אי-סימטרי, פיצול ואיחוד. אחר כך להתקין CORE ולחזור על זה
4: זמן ולוגים	לוגים מובנים ב-JSON במדמה ובבדיקות, הזרקת סטיית שעות עם libfaketime, ומיזוג ציר זמן ב-pandas שמחשב זמן התכנסות
לאורך זמן	לקרוא את תבניות STP/STD/STR ולמפות עליהן את מסמכי הצוות לתייג כל בדיקה במזהה דרישה

פרויקט צד מהנה ובעל ערך גבוה: PyTAK + FreeTAKServer + שני מרחבי שמות – לשלוח CoT ב-multicast, לשבור, לתקן, ולבדוק תפוגה.

סדר עדיפויות מעודכן

ICD + parsing → Management plane → Time sync → Multi-node emulation → Defense test process
→ Multicast → Sockets & reliability → Soak & chaos → GUI automation → Model-based testing

הסתייגויות למידע בפרקים 13-27

- כל המידע על BNET מגיע מחומר פומבי של רפאל ומהתקשורת הביטחונית. **אין מידע פומבי** על כך ש-BNET או אפליקציית הניטור שלו משתמשים ב-SNMP, CoT, ב-NTP/PTP או בפורמט מסוים. יש למפות את המושגים הכלליים על ה-ICD הפנימי ולא להניח.
- תוכן ניהול המפתחות הוא עקרוני בכוונה. יש לפעול לפי נוהלי הביטחון של רפאל בכל הנוגע לחומר מפתח אמיתי או לתצורה מסווגת.

- כמה תקנים הם יעד נע: אשרור STANAG 5630 מהדורה 2 ברבעון האחרון של 2026 הוא תחזית; ל-MIL-STD-461 יש מהדורה H חדשה מ-G; MIL-STD-498 בוטל רשמית אף שתבניות המסמכים שלו נותרו בשימוש.
- מצב הכלים משתנה. כדאי לוודא את מצב WinAppDriver ואת גרסת FlaUI לפני החלטה על תקן צוותי.
- מספרי הדיוק של NTP ו-PTP הם ערכים טיפוסיים, והתוצאה בפועל תלויה בתמיכת חומרה בחותמות זמן ובעומס הרשת.